BÁO CÁO NGHIÊN CỨU & THỰC NGHIỆM

CAR PRICE AI

DỰ ĐOÁN GIÁ XE Ô TÔ ĐÃ QUA SỬ DỤNG BẰNG CATBOOST REGRESSOR

Phiên bản: tổng hợp từ mã nguồn, CSV và model thực tế trong ZIP được cung cấp
Đơn vị dự báo: triệu VNĐ / giá niêm yết trên tin đăng
Ngày tổng hợp: 25/10/2026

30.652	Tin đăng trong dữ liệu gốc
24.582	Tin xe cũ dùng làm dữ liệu học
0,9568	R^2 trên Test
112,08	MAE Test (triệu VNĐ)

Nguồn nội bộ: **Old_Car_Price_AI_zip(1).zip** - gồm dữ liệu gốc, mã nguồn Python, mô hình .cbm và các báo cáo kiểm thử. Tham khảo bên ngoài được liệt kê riêng ở cuối tài liệu.

TÓM TẮT ĐIỀU HÀNH

Car_Price_AI là hệ thống **hồi quy có giám sát** dự đoán giá niêm yết xe ô tô đã qua sử dụng tại Việt Nam. Dữ liệu gốc có 30.652 bản ghi, 21 cột; quá trình lọc **"Xe đã dùng"** giữ lại 24.582 bản ghi. Sau biến đổi, bộ dữ liệu có 14 đặc trưng và một biến đích **price_million**.

Nhóm phát triển sử dụng **CatBoostRegressor** nhờ khả năng làm việc với biến phân loại như hãng, mẫu xe, loại thân xe, hộp số; hỗ trợ NaN ở đặc trưng số; và khả năng học quan hệ phi tuyến trên dữ liệu bảng. Đây là **lựa chọn có cơ sở kỹ thuật**, không phải kết luận mô hình tốt nhất từ một cuộc thi thuật toán trên cùng dữ liệu.

Thực nghiệm: Train 17.207, Validation 3.687, Test 3.688; mô hình có R^2 Test = 0,9568; MAE = 112,08 triệu; RMSE = 376,70 triệu. 85,30% tin Test có sai số tuyệt đối không quá 20% giá niêm yết. Chỉ số 85,30% này là **tỷ lệ nằm trong ngưỡng sai số ±20%**, không phải classification accuracy.

Rủi ro trọng tâm: 418 tin trên 2 tỷ đồng (11,3% Test) đóng góp 50,1% tổng sai số tuyệt đối. Có 15.427 chuỗi chỉ ghi "L/100Km" không kèm trị số trong dữ liệu xe cũ, nhưng code quy ước thành 1,0; đây là quyết định xử lý dữ liệu chưa được chứng thực về mặt ngữ nghĩa. Số km có giá trị cực đại 4.294.967.295, gợi ý lỗi/giá trị thay thế cần rà soát. Các kết luận không thể suy rộng mặc nhiên sang giá giao dịch hay thị trường 2026.

GHI CHÚ KỸ THUẬT

Tất cả số đo thực nghiệm trong báo cáo đến từ CSV và các file **reports/** nằm trong ZIP; thông tin lý thuyết lấy từ bài báo CatBoost NeurIPS 2018 và tài liệu chính thức. Không huấn luyện lại mô hình hoặc nhập dữ liệu ngoài để "nâng" chỉ số.

Mục lục nội dung

01 Bối cảnh, mục tiêu và phạm vi

02 Dữ liệu nguồn và đặc trưng

03 Tiền xử lý, chất lượng dữ liệu

04 Nền tảng toán học và CatBoost

05 Kiến trúc mã nguồn và thứ tự chạy

06 Thiết kế thực nghiệm và chỉ số

07 Kết quả, hình ảnh hóa và phân tích lỗi

08 Giới hạn, cải tiến, kết luận

09 Phụ lục, nguồn tham khảo

1. BỐI CẢNH, MỤC TIÊU VÀ PHẠM VI

1.1. Bài toán thực tiễn

Giá xe cũ chịu ảnh hưởng của hãng, dòng xe, năm sản xuất, quãng đường đã đi, cấu hình động cơ, trang bị và điều kiện thị trường. Người mua, người bán và người nghiên cứu cần một công cụ ước tính có thể lặp lại, giảm phụ thuộc vào nhận định chủ quan.

Mục tiêu của Car_Price_AI là xây dựng hàm hồi quy **f(x) giá niêm yết**, trong đó x là thông số chiếc xe. Sản phẩm bao gồm chương trình dòng lệnh để nhập cấu hình, mô hình đã huấn luyện và các file kiểm thử được lưu lại.

1.2. Câu hỏi nghiên cứu

• Các đặc trưng nào đóng vai trò lớn trong dự báo theo mô hình CatBoost?

• Độ lệch giữa dự báo và giá đăng bán là bao nhiêu trên tập Test độc lập?

• Độ chính xác thay đổi thế nào giữa xe phổ thông, xe giá cao và các hãng khác nhau?

• Những rủi ro dữ liệu nào làm giảm độ tin cậy, và có thể khắc phục bằng quy trình nào?

1.3. Ranh giới và giả định

Biến cần dự đoán: giá ghi trên tin đăng, đã quy đổi về **triệu VNĐ**. Không có nhãn giá giao dịch cuối cùng; do đó không gọi đầu ra là "giá thị trường chính xác" hay "giá thanh khoản". Dữ liệu gốc ghi năm sản xuất tới năm 2023 và không có bằng chứng bảo đảm tương thích nguyên trạng với diễn biến giá năm 2026.

Phạm vi triển khai: Python, pandas, scikit-learn để chia và đánh giá tập dữ liệu; CatBoost để học; chương trình Terminal để dự đoán. Không có hệ thống thu thập dữ liệu trực tuyến hay giám sát drift tự động trong mã nguồn hiện tại.

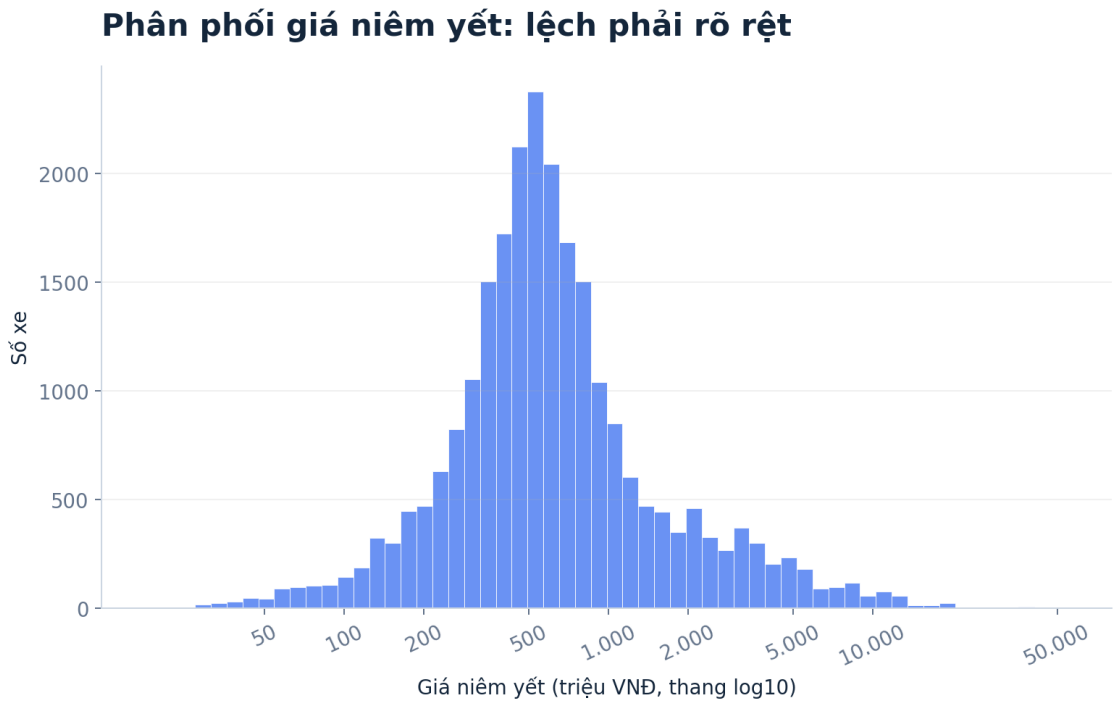
1.4. Tham khảo học thuật

Bài báo CatBoost của Prokhorenkova và cộng sự (NeurIPS 2018) trình bày ordered target statistics và ordered boosting. Một nghiên cứu tiếng Việt về dự báo giá xe cũ bằng hồi quy tuyến tính (Tạp chí Khoa học Tài nguyên và Môi trường, 2025) gợi mở khung trình bày dữ liệu - thuật toán - chỉ số - giới hạn; khác dữ liệu và split nên không so trực tiếp R^2 giữa hai nghiên cứu. [R1], [R6]

2. DỮ LIỆU VÀ MÔ TẢ ĐẶC TRƯNG

2.1. Từ dữ liệu raw đến dữ liệu cho CatBoost

Chỉ tiêu	Giá trị thực tế
File nguồn	data/raw/car_detail.csv
Kích thước gốc	30.652 dòng × 21 cột
Xe mới / xe đã dùng	6.070 / 24.582 dòng
Dữ liệu sau xử lý	24.582 dòng × 15 cột
Bài toán	14 input features price_million
Số hãng / mẫu xe sau làm sạch	74 hãng / 460 giá trị Grade
Đơn vị giá	Triệu đồng Việt Nam



Hình 1. Phân phối giá niêm yết có đuôi phải dài; trục hoành log10 để quan sát toàn dải.

Trung vị giá là **555 triệu**, trong khi trung bình khoảng **1.069,73 triệu**; khoảng 95% mẫu có giá không quá **3.850 triệu**. Giá lớn nhất trong tập sạch là **54.000 triệu**. Cấu trúc này lý giải vì sao RMSE nhạy mạnh với một nhóm nhỏ xe giá rất cao.

2.2. 14 đặc trưng đầu vào

Feature	Loại	Nội dung	Ghi chú
brand	Phân loại	Hãng xe	Không rõ nếu trống
grade	Phân loại	Tên mẫu xe / Grade	Không mặc định là phiên bản/trim
body_type	Phân loại	Dòng xe	Chuẩn hóa khoảng trắng
origin	Phân loại	Xuất xứ	Giữ nhãn nguồn
transmission	Phân loại	Hộp số	Giữ nhãn nguồn
drivetrain	Phân loại	Hệ dẫn động	Giữ nhãn nguồn
fuel_type	Phân loại	Nhiên liệu tách từ Động cơ	Xăng, Dầu, Hybrid, Điện…
engine_liters	Số	Dung tích động cơ	Trích giá trị L; có NaN
doors	Số	Số cửa	Trích chữ số
seats	Số	Số chỗ	Trích chữ số
fuel_consumption	Số	Lít / 100 km	Chuỗi đơn vị-only 1.0
year	Số	Năm sản xuất	Có NaN
mileage	Số	Km đã đi	Bỏ dấu phân tách hàng nghìn
mileage_is_zero	Nhị phân	Cờ bằng 0 km	Không đồng nghĩa chắc chắn xe chưa chạy

Các trường tên xe, mô tả, URL, hai màu và “Hệ thống nạp nhiên liệu” không dùng làm feature. “Mã tin” và “Tình trạng” được sử dụng ở bước quản lý/lọc dữ liệu, nhưng không đi vào X huấn luyện.

3. TIỀN XỬ LÝ VÀ KIỂM SOÁT CHẤT LƯỢNG

3.1. Quy trình làm sạch hiện tại

• Đọc CSV raw bằng pandas; kiểm tra kiểu dữ liệu, giá trị thiếu và tình trạng xe.

• Chỉ giữ Tình trạng == "Xe đã dùng"; loại dòng trùng hoàn toàn bằng drop_duplicates().

• Chuyển giá "1 Tỷ 350 Triệu" 1.350 (triệu); số km "60,000 Km" 60.000.

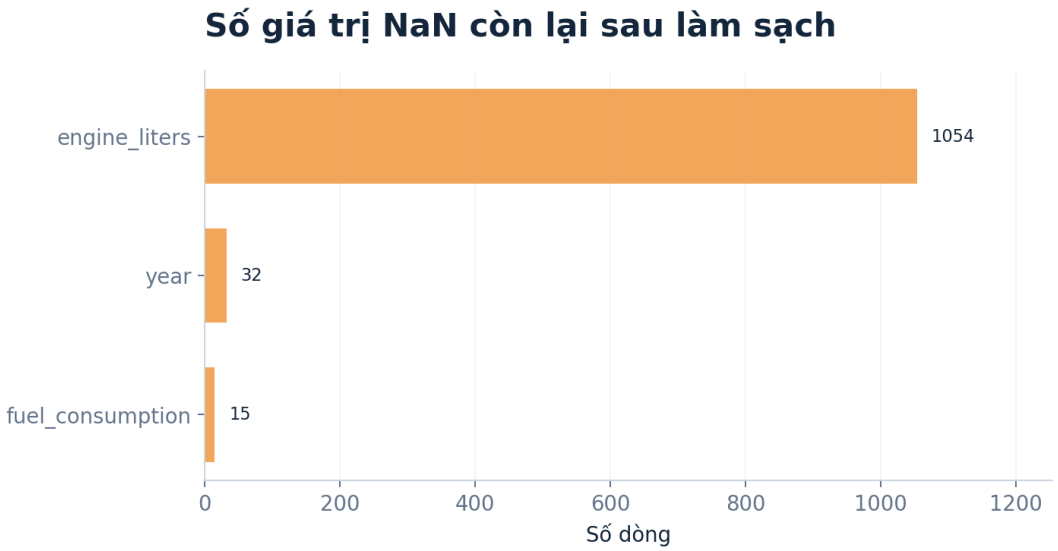
• Tách chuỗi Động cơ thành fuel_type và engine_liters; chuẩn hóa doors, seats, year và fuel_consumption.

• Chuyển category rỗng thành "Không rõ"; giữ NaN của feature số để CatBoost tự xử lý.

• Loại bản ghi không có target hợp lệ; xuất cars_clean_v2.csv.

3.2. NaN còn lại không đồng nghĩa lỗi pipeline

Cột	NaN / 24.582	Đánh giá
engine_liters	1.054 (4,29%)	Thiếu thông số dung tích
year	32 (0,13%)	Năm không chuyển được thành số
fuel_consumption	15 (0,06%)	Chỉ phản ánh NaN sau quy đổi, không phản ánh thiếu ý nghĩa
Các cột khác	0	Category được gán "Không rõ", số đọc được hoặc cờ nhị phân



Hình 2. NaN kỹ thuật còn lại trong bản dữ liệu sau làm sạch.

3.3. Vấn đề trọng yếu: "L/100Km" bị giả định thành 1,0

Trong raw dành cho xe cũ, có **15.427** giá trị sau khi trim chỉ chứa "L/100Km" mà không kèm con số. Hàm parse_fuel_consumption() hiện tại **gán các chuỗi này thành 1,0**; tổng cộng 15.457 bản ghi có fuel_consumption == 1 sau xử lý (bao gồm cả chuỗi ghi rõ 1).

GHI CHÚ KỸ THUẬT

“L/100Km” thuần túy biểu thị đơn vị, không chứng minh tiêu hao = 1 lít / 100 km. Việc gán 1,0 là **quy ước dự án theo người xây dựng**, không phải diễn giải khách quan. Khoảng 62,8% dữ liệu có thể chịu tác động của quy ước này. Không sửa kết quả đã huấn luyện trong báo cáo; đề xuất thực nghiệm đối chứng về sau.

3.4. Điểm bất thường khác cần rà soát

Có **124** bản ghi có mileage trên 1.000.000 km và giá trị lớn nhất là **4.294.967.295 km** (trùng 2^{32} 1), nhiều khả năng là giá trị sentinel/lỗi dữ liệu hoặc bất thường cần xác minh; chưa chứng minh tất cả 124 bản ghi đều sai. File preprocess.py có nhánh **return np.na** (không tồn tại trong numpy) khi giá là chuỗi rỗng hoặc “Liên hệ”; nhánh này không ảnh hưởng các bản ghi đã xử lý thành công nhưng cần sửa thành np.nan trước khi chạy lại trên dữ liệu mới.

4. CATBOOST: CƠ SỞ VÀ LÝ DO LỰA CHỌN

4.1. CatBoost là gì?

CatBoost là một framework Gradient Boosting Decision Trees (GBDT), nổi bật ở xử lý biến phân loại. Hệ thống xây dựng một tập hợp cây nhỏ theo tuần tự; mỗi cây mới học từ phần sai số / gradient của các cây trước. Dự đoán cuối cùng là tổng có trọng số của các cây. [R1]

4.2. Tại sao phù hợp với Car_Price_AI?

• Dữ liệu bảng pha trộn 7 category (brand, grade, transmission...) và 7 đặc trưng số/nhị phân.

• Có category nhiều giá trị (460 mẫu Grade), khiến one-hot thủ công có thể phình số cột và khó quản lý.

• Khả năng học tương tác phi tuyến như hãng × đời xe × dung tích máy × số km.

• Xử lý NaN ở numerical features ngay trong thuật toán; không phải fit bộ điền thiếu từ toàn bộ dữ liệu.

• Cây đối xứng mặc định giúp phép suy luận dự đoán nhanh, dễ triển khai từ file model .cbm.

4.3. CatBoost so với các họ thuật toán khác (so sánh lý thuyết)

Thuật toán	Đặc điểm / trở ngại ở dataset này
Linear/Ridge	Đơn giản, dễ giải thích hệ số; cần encoding, khó nắm bắt tương tác phi tuyến nếu không tạo features.
Random Forest	Mạnh với quan hệ phi tuyến; dùng sklearn cần mã hóa category và xử lý missing phù hợp; không học boosting tuần tự.
XGBoost / LightGBM	Các GBDT mạnh; cần cấu hình riêng để xử lý categorical/missing theo từng thư viện.
CatBoost	Native categorical handling; thống kê category có kiểm soát rò rỉ và xử lý numerical NaN; phù hợp ưu tiên ít bước encoding.

Giới hạn kết luận: Không có benchmark công bằng giữa các thuật toán trên cùng split của Car_Price_AI. Không được diễn giải bảng này thành "CatBoost chắc chắn chính xác nhất". Lựa chọn là quyết định thiết kế dựa trên cấu trúc dữ liệu và tài liệu [R1-R4].

4.4. Một phân biệt quan trọng: Ordered statistics Ordered boosting luôn được bật

Theo tài liệu, CatBoost có cơ chế thống kê thứ tự cho category và thuật toán ordered boosting để giảm prediction shift. Tuy nhiên, **model trong ZIP được kiểm tra trực tiếp** và tham số hiệu lực cho thấy **boosting_type = Plain** trên CPU; grow_policy = SymmetricTree; nan_mode = Min. Vì thế báo cáo không khẳng định phiên huấn luyện hiện tại đã bật Ordered boosting. [R1], [R4]

5. NỀN TẢNG TOÁN HỌC

5.1. Phát biểu bài toán hồi quy

Với x_i X mô tả một chiếc xe và y_i ℝ là giá niêm yết theo triệu VNĐ, ta tìm hàm F(x) sao cho $\hat{y}_i = F(x_i)$. Hàm tổn thất trong code huấn luyện là RMSE (tương đương tối ưu theo MSE dưới căn bậc hai đơn điệu trong đánh giá tổng quát).

Gradient boosting (mô hình cộng): $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, với h_m là cây ở vòng m và η = learning_rate = 0,05. Mỗi cây học xu hướng sửa phần dự đoán hiện tại. Công thức là diễn giải GBDT mức khái quát, không thay thế toàn bộ chi tiết tối ưu của CatBoost.

Với squared error: $L = (1/n) \Sigma_i (y_i - \hat{y}_i)^2$. Gradient theo $\hat{y}_i$: $L/\hat{y}_i = 2(\hat{y}_i - y_i)/n$; hướng giảm gradient tỷ lệ với $(y_i - \hat{y}_i)$.

5.2. Thống kê category với prior, kiểm soát leakage

CatBoost tạo biến số từ category bằng thống kê tính trên những hàng xuất hiện **trước hàng đang xét trong một hoán vị**, thay vì lấy trực tiếp target trung bình của toàn bộ tập có chứa chính hàng đó. Một **minh họa CTR dạng nhị phân** là:

$$\mathbf{CTR_i(c) = (\Sigma_j{:}\pi(j)<\pi(i),\ x_j=c\ t_j + a) / (N_i(c) + 1)}$$

Trong đó t_j là nhãn nhị phân/nhãn đã bucket hóa; a là prior; $N_i(c)$ là số mẫu cùng category xuất hiện trước quan sát i. Với hồi quy thực tế, CatBoost có thể lượng tử hóa target và dùng nhiều kiểu CTR; công thức trên **là ví dụ về nguyên lý**, không phải tái hiện đúng mọi loại CTR nội bộ. [R2]

5.3. Cây đối xứng (oblivious / symmetric trees)

Mỗi tầng của cây sử dụng một phép chia giống nhau trên mọi node cùng tầng. Với depth = 6, một cây đầy đủ có thể có tối đa 2^6 = **64 lá**. Nhiều cây như vậy được kết hợp để mô tả quan hệ phi tuyến. Cơ chế tree structure trong model kiểm tra được là SymmetricTree. [R4]

5.4. Early stopping

Code cho phép tối đa 2.000 vòng; dùng Validation MAE và dừng nếu không cải thiện sau 100 vòng. Thực tế model lưu **2.000 cây** và best_iteration = **1.999**. Nghĩa là không có bằng chứng early stopping đã cắt bớt số vòng; mô hình đang tốt nhất (theo Validation) tại giới hạn đã đặt. Không thể khẳng định 2.000 vòng là tối ưu tuyệt đối.

6. KIẾN TRÚC DỰ ÁN VÀ DÒNG CHẢY FILE

Thứ tự vận hành: data_check.py (khám phá) preprocess.py (làm sạch) train.py (huấn luyện, lưu mô hình) evaluate.py (phân tích sau train) run.py (dự đoán qua Terminal). evaluate.py và run.py là hai nhánh tiêu thụ artifacts sau train; không bắt buộc chạy evaluate trước khi sử dụng run.

File chạy	Input	Output / chức năng
data_check.py	data/raw/car_detail.csv	In shape, NaN, dtypes, trùng, tình trạng; không ghi file mới
preprocess.py	data/raw/car_detail.csv	data/processed/cars_clean_v2.csv
train.py	data/processed/cars_clean_v2.csv	models/*.cbm, models/feature_columns.json, reports/metrics.json, feature_importance.csv, test_predictions.csv
evaluate.py	CSV đã xử lý + model + metrics + test_predictions	reports/dataset_comparison.csv, errors_by_brand.csv, errors_by_price_segment.csv, top_20_errors.csv, evaluation_summary.json
run.py	model .cbm + feature_columns.json + processed CSV	Nhập xe và xuất giá dự đoán trên Terminal (không tạo file mới)

6.1. Vai trò của thư mục artifacts

• **data/raw/:** lớp dữ liệu nguồn, không sửa trực tiếp; bảo toàn khả năng tái tạo.

• **data/processed/:** dữ liệu đã đổi tên, trích numeric, sẵn sàng đưa vào model.

• **models/:** mô hình .cbm và metadata về thứ tự/tên features để inference tương thích.

• **reports/:** bằng chứng thực nghiệm, phân tích lỗi và tầm quan trọng của từng feature.

6.2. Danh mục hàm trong preprocess.py

Hàm	Công việc chính
parse_price	Đọc giá Tỷ + Triệu price_million; cần sửa nhánh np.na
parse_mileage	Đọc “60,000 Km” 60000.0
parse_fuel_type	Tách Xăng/Dầu/Hybrid/Điện từ chuỗi Động cơ
parse_engine_liters	Regex nhận dung tích dạng “2.0 L”
parse_count	Trích số cửa, số chỗ
parse_fuel_consumption	Đổi “7 L/100Km” 7; “L/100Km” 1 theo quy ước hiện có
clean_category	Xóa khoảng trắng thừa, thay category trống bằng “Không rõ”

7. THÀNH PHẦN MÃ NGUỒN: HUẤN LUYỆN, ĐÁNH GIÁ, DỰ ĐOÁN

7.1. train.py - mô hình hóa

File train.py chạy chủ yếu ở phạm vi module, không định nghĩa hàm riêng. Thứ tự: đọc CSV; X/y; liệt kê 7 categorical features; chuyển category thành str và số thành numeric; chia 70/15/15; khai báo CatBoostRegressor; fit(...eval_set, early_stopping_rounds=100); predict Test; tính MAE/RMSE/R^2; tính Feature Importance; lưu model và report.

Tham số thực tế	Giá trị
iterations	2000
depth	6
learning_rate	0,05
loss_function	RMSE
eval_metric	MAE
l2_leaf_reg	5
random_seed	42
boosting_type hiệu lực	Plain (CPU)
grow_policy / NaN	SymmetricTree / Min

7.2. evaluate.py - kiểm chứng sau huấn luyện

Hàm **calculate_metrics(y_true, y_pred)** trả MAE, RMSE, R^2. Phần còn lại nạp lại CSV và model, tái lập cách chia với random_state=42, dự đoán trên Train và Validation, dùng metric Test đã lưu, tạo bảng sai số theo hãng/phân khúc và chọn 20 dự đoán lệch lớn nhất.

Lưu ý: kiểm tra kích thước split bằng train_samples chỉ bắt được thay đổi số dòng, **không chứng minh thứ tự hay nội dung CSV chưa thay đổi**. Tập Test đã được dùng để phân tích lỗi nên không nên tiếp tục tune nhiều vòng theo Test rồi xem đó là chỉ số khách quan cuối cùng.

7.3. run.py - suy luận thực tế

Hàm	Vai trò
clean_category / parse_number	Làm sạch category / ép kiểu số nhập từ bàn phím
parse_mileage / parse_fuel_consumption	Nhận định dạng km và tiêu hao tương thích logic xử lý
ask_number / ask_mileage / ask_fuel_consumption	Tương tác CLI và kiểm tra ràng buộc nhập số
ask_category	Hiển thị toàn bộ category tìm thấy trong CSV, chọn bằng số/tên
load_model	Đọc .cbm và feature_columns.json
input_car	Gom các input thành dict của xe; thêm hai cờ mileage
prepare_input	Sắp cột theo feature_columns.json; cast lại loại dữ liệu
predict_price	Gọi model.predict(), chặn giá âm về 0
main	Điều khiển vòng nhập nhiều xe và in dự đoán

Lưu ý: run.py tạo **mileage_missing** nhưng model trong ZIP chỉ có 14 features, không bao gồm cột đó; **prepare_input()** chỉ lấy các features trong JSON nên cờ thừa bị bỏ qua. Danh sách category lấy toàn bộ dữ liệu đã làm sạch, nhưng chưa ràng buộc tổ hợp hãng - mẫu - cấu hình; người dùng vẫn có thể chọn tổ hợp không thực tế.

8. THIẾT KẾ THỰC NGHIỆM VÀ CHỈ SỐ

8.1. Chia dữ liệu

Tập	Số bản ghi	Tỷ trọng	Vai trò
Train	17.207	70,0%	Học tham số, thống kê category
Validation	3.687	15,0%	Đánh giá vòng train, early stopping
Test	3.688	15,0%	Đánh giá bên ngoài Train/Validation

Phân chia ngẫu nhiên random_state=42, không chia theo thời gian, giá hay nhóm mẫu xe. Việc rải xe hiếm không đều giữa tập Train và Test có thể tạo sai số cực lớn. Nên cân nhắc stratification theo dải giá và grouped/temporal holdout cho nghiên cứu tiếp theo.

8.2. Các công thức đánh giá

MAE = (1/n) Σ_i $|y_i - \hat{y}_i|$ - sai số tiền tệ trung bình, đơn vị triệu VNĐ.

RMSE = $[(1/n) \Sigma_i (y_i - \hat{y}_i)^2]$ - nhạy hơn với những lỗi lớn.

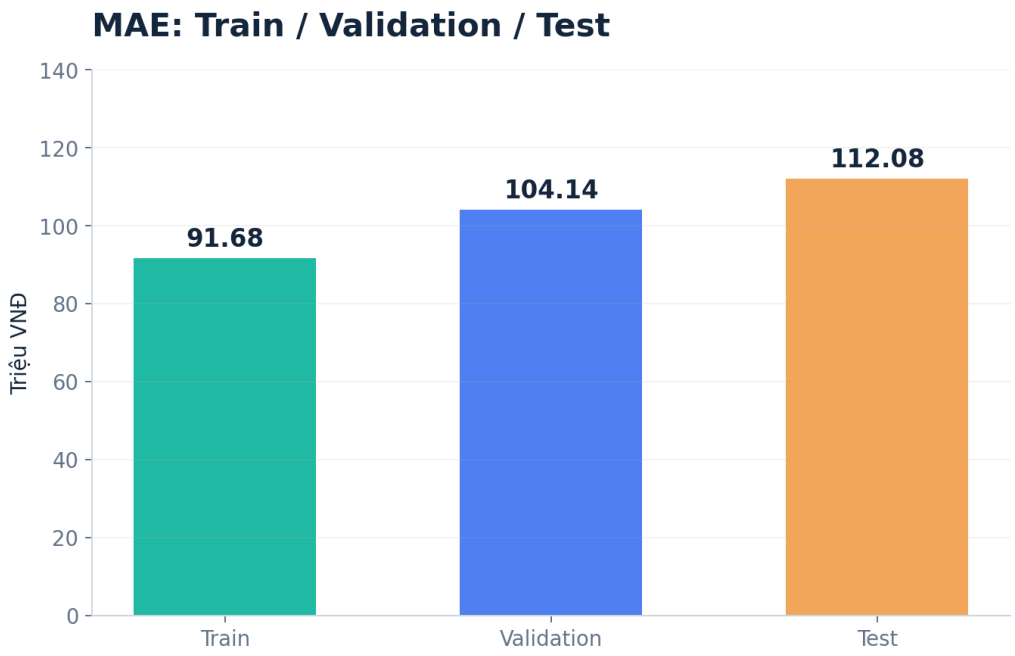
$R^2 = 1 - [\Sigma_i (y_i - \hat{y}_i)^2] / [\Sigma_i (y_i - \bar{y})^2]$ - tỉ lệ biến thiên nhãn được giải thích tương đối với dự đoán trung bình; R^2 có thể âm.

MAPE = $(100/n) \Sigma_i |(y_i - \hat{y}_i)/y_i|$ - sai số phần trăm trung bình với $y_i > 0$.

Accuracy theo ngưỡng ±20% (không phải accuracy phân loại): $A_{20} = (1/n) \Sigma_i 1\{|y_i\hat{y}_i| / y_i \le 0{,}20\}$. Đây là một thước đo vận hành tự định nghĩa để diễn giải "bao nhiêu tin được dự đoán trong sai số 20%".

8.3. Kết quả tổng thể thực nghiệm

Chỉ số	Train	Validation	Test
Số mẫu	17.207	3.687	3.688
MAE (triệu VNĐ)	91,68	104,14	112,08
RMSE (triệu VNĐ)	245,62	251,22	376,70
R^2	0,9831	0,9800	0,9568



Hình 3. Validation MAE cao hơn Train 12,46 triệu (1,14 lần).

9. KẾT QUẢ KIỂM THỬ VÀ PHÂN TÍCH SAI SỐ

9.1. Một con số “accuracy” dễ hiểu, nhưng phải định nghĩa rõ

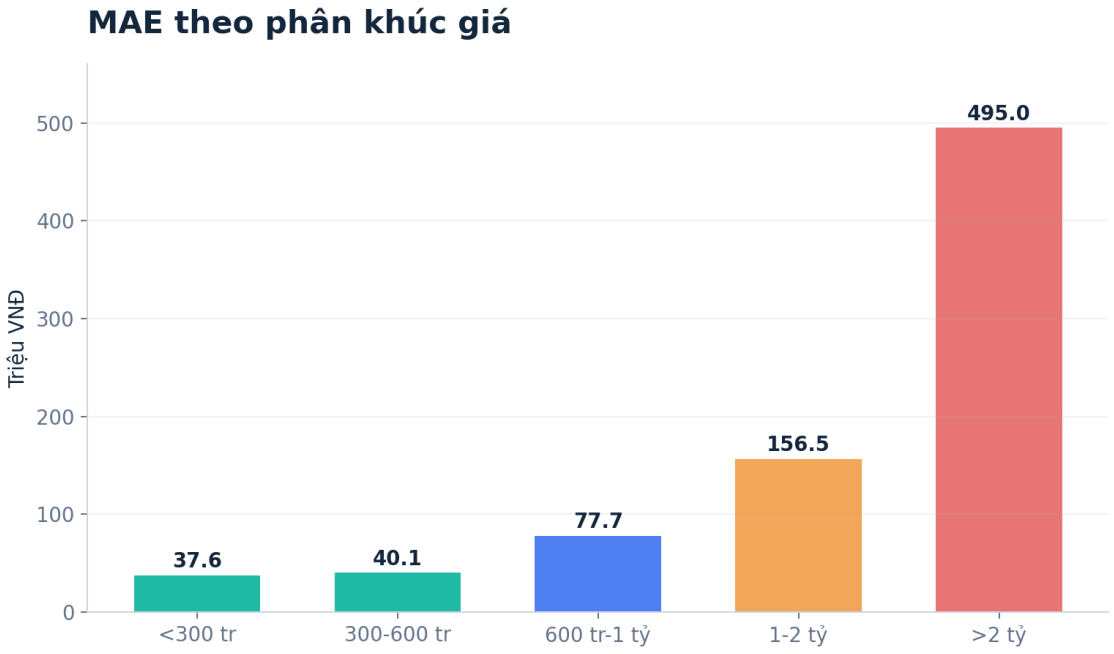
R^2 = 0,9568 không đồng nghĩa “độ chính xác 95,68%”. Với hồi quy, accuracy thông thường không có một định nghĩa duy nhất. Từ test_predictions.csv, tính tỷ lệ dự đoán nằm trong sai số tương đối:

Ngưỡng sai số tuyệt đối tương đối	Tỷ lệ mẫu Test đạt ngưỡng
5%	39,18%
10%	63,80%
15%	77,25%
20%	85,30%
25%	89,34%
30%	92,57%

MAPE tổng thể trên Test = **12,66%**; sai số phần trăm trung vị khoảng **6,93%**; WAPE (tổng sai số tuyệt đối / tổng giá niêm yết) khoảng **10,65%**. Các chỉ số % nhạy với nhóm xe giá rẻ và cần được dùng kèm MAE theo đơn vị tiền.

9.2. Hiệu quả theo phân khúc

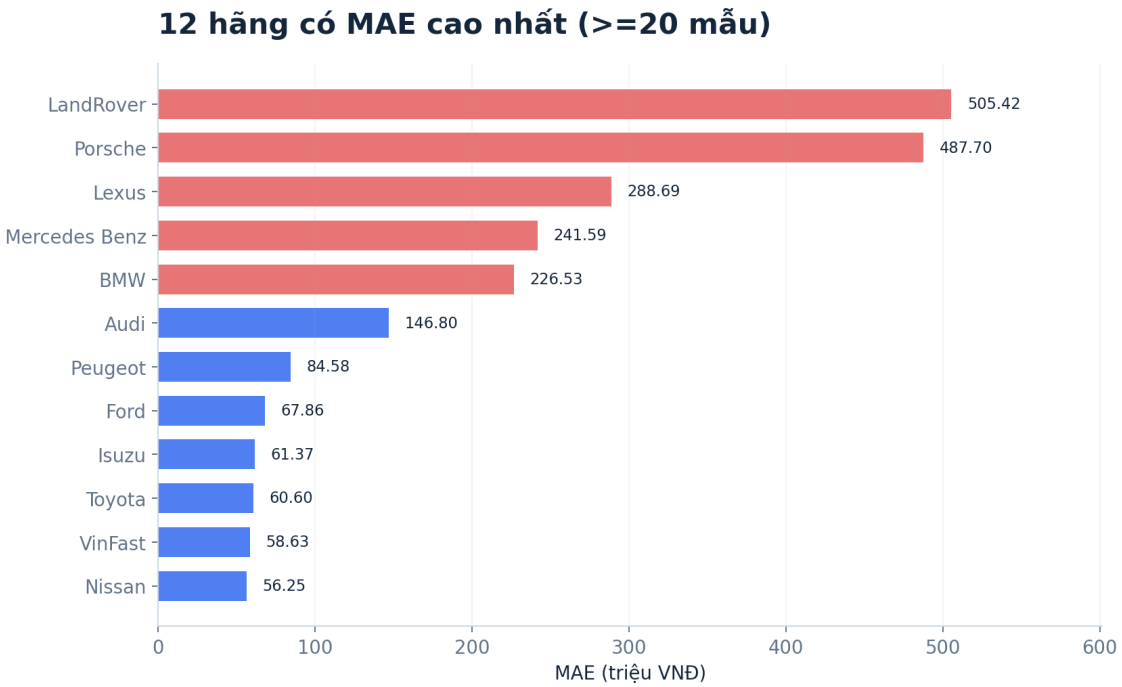
Phân khúc	Số mẫu	MAE (triệu)	MAPE (%)
Dưới 300 tr	694	37,58	25,54
300-600 tr	1377	40,08	8,90
600 tr-1 tỷ	793	77,75	10,01
1-2 tỷ	406	156,48	11,30
Trên 2 tỷ	418	494,97	9,98



Hình 4. MAE tăng rất mạnh ở nhóm xe giá niêm yết trên 2 tỷ.

Nhóm **trên 2 tỷ** có 418/3.688 mẫu (11,3% Test) nhưng đóng góp **50,1%** tổng sai số tuyệt đối. Tuy nhiên MAPE của nhóm này khoảng 9,98%, thấp hơn nhóm dưới 300 triệu (25,54%). Do đó phải đánh giá cả đơn vị tiền và phần trăm.

10. PHÂN TÍCH SAI SỐ THEO HÃNG VÀ TỪNG XE



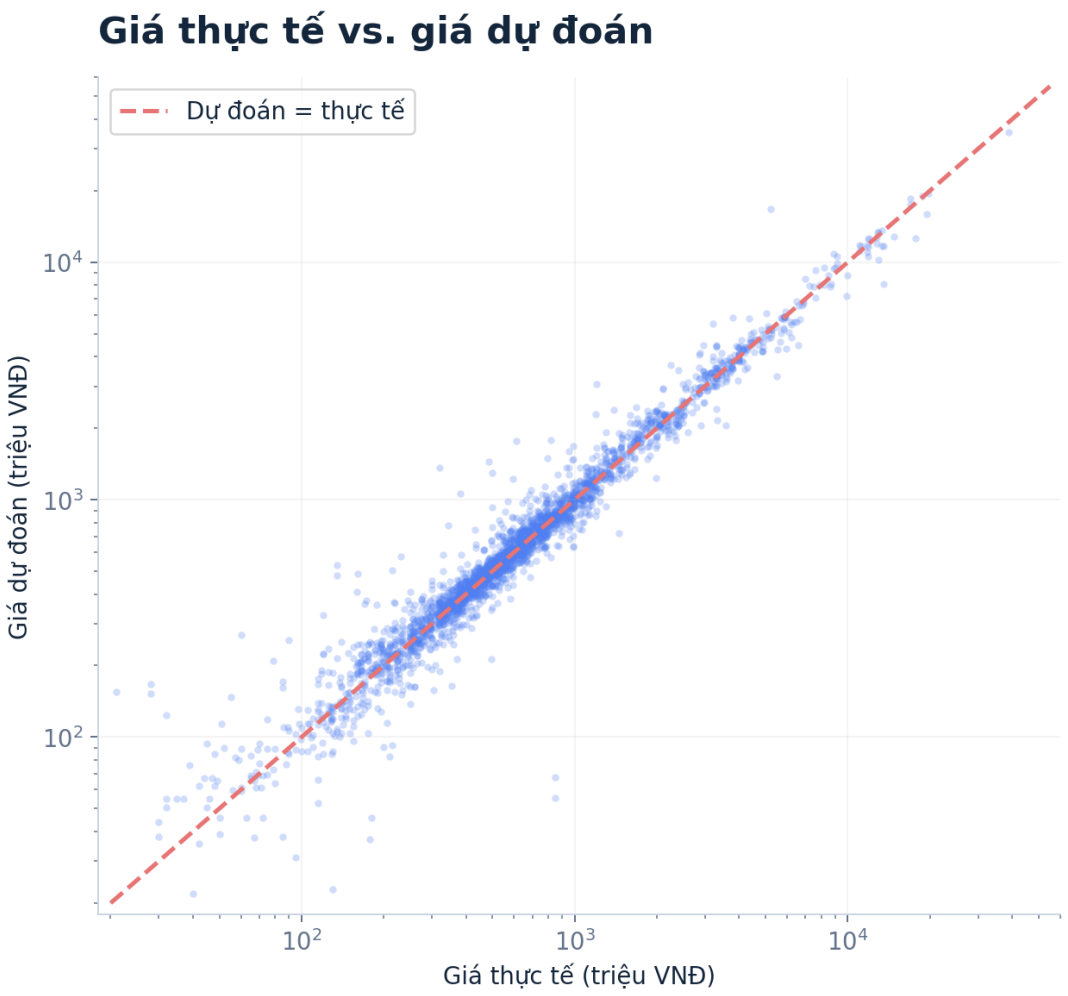
Hình 5. MAE theo các hãng đủ 20 bản ghi Test, ưu tiên mô tả độ tin cậy mẫu.

Ví dụ trong nhóm có từ 20 mẫu Test: LandRover có MAE 505,42 triệu (52 mẫu), Porsche 487,70 triệu (51), Lexus 288,69 triệu (125), Mercedes Benz 241,59 triệu (405). Các hãng số lượng nhỏ có độ bất định cao; MAE lớn một phần vì mặt bằng giá xe cao.

10.1. Những trường hợp dự đoán lệch mạnh

Xe	Giá thật	Dự đoán	|Sai số|
RAM 1500 (2022)	5.200	16.796	11.596
Mercedes G class (2014)	10.999	4.144	6.855
Bentley Mulsanne (2020)	29.500	36.177	6.677
Bentley Bentayga (2019)	13.500	8.056	5.444
Lamborghini Urus (2019)	17.700	12.570	5.130

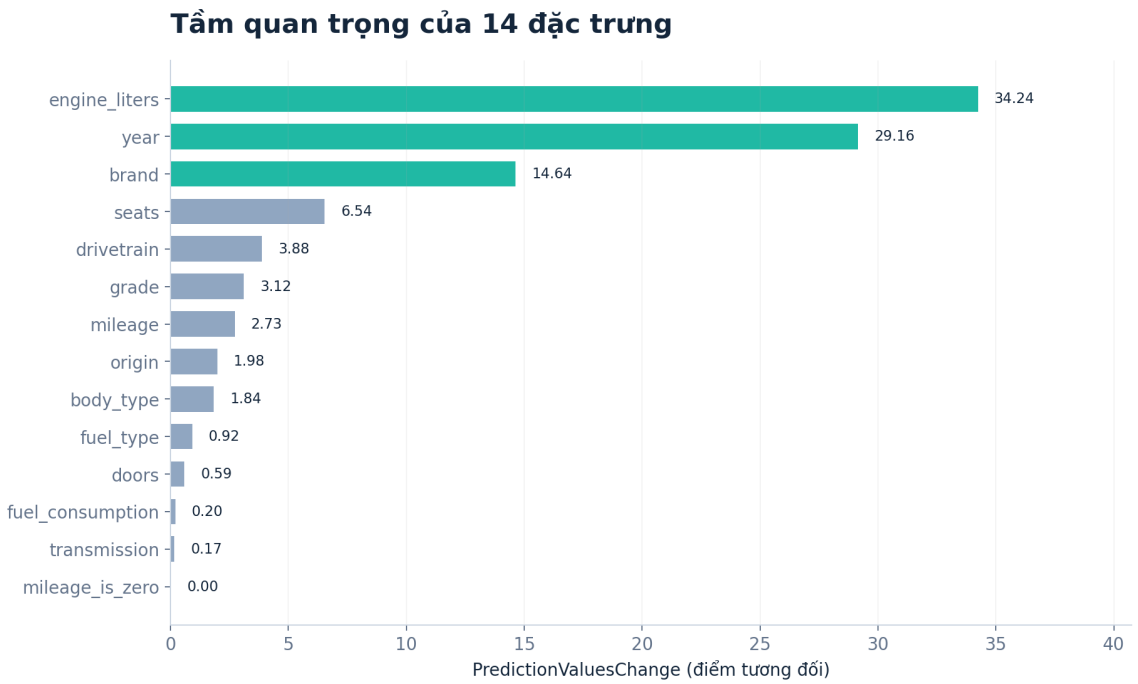
Đơn vị bảng: triệu VNĐ. RAM 1500 là outlier nghiêm trọng theo sai số tuyệt đối, nhưng không đủ thông tin để khẳng định tin đăng đó là dữ liệu sai. Có thể cần kiểm tra phiên bản, tình trạng, lịch sử xe, sai lệch niêm yết, độ hiếm và mẫu học cùng loại.



Hình 6. Dự đoán so với giá niêm yết Test trên trục log; đường đỏ là dự đoán hoàn hảo.

11. MỨC ĐỘ QUAN TRỌNG CỦA ĐẶC TRƯNG

File **reports/feature_importance.csv** lấy từ CatBoost **get_feature_importance()**; đối với tác vụ hồi quy thông thường, đây là độ quan trọng kiểu PredictionValuesChange. Đây là đóng góp vào thay đổi dự đoán theo cấu trúc model, **không chứng minh quan hệ nhân quả** hay độ chính xác độc lập của từng cột. [R5]



Hình 7. 14 đặc trưng xếp theo tầm quan trọng; tổng các điểm xấp xỉ 100.

Nhóm nổi bật	Importance
engine_liters	34,24
year	29,16
brand	14,64
seats	6,54
drivetrain	3,88
grade	3,12
mileage	2,73
fuel_consumption	0,20

Ba biến hàng đầu cộng lại chiếm khoảng **78,03%** tầm quan trọng của mô hình theo thước đo này. Đặc biệt, engine_liters cao hơn brand và grade không có nghĩa dung tích động cơ "gây ra" mọi biến động giá: nó có thể tương quan với phân khúc và hãng xe. fuel_consumption chỉ 0,20 điểm nhưng không đủ để chứng minh quy ước gán 1,0 hoàn toàn vô hại.

12. GIỚI HẠN, KHẮC PHỤC LỖI VÀ ĐỊNH HƯỚNG

12.1. Kiểm thử cuối cùng đã phát hiện điều gì?

Phát hiện	Rủi ro	Đề xuất xử lý
Mẫu cực hiếm/xe sang sai hàng tỷ	Dải giá rộng, ít mẫu, thiếu trim cụ thể	Bổ sung dữ liệu cùng thị trường; phân tích lỗi riêng theo hãng và phân khúc
"L/100Km" 1,0 cho nhiều dòng	Đặc trưng được gán giá trị không đủ căn cứ	Thử giữ NaN hoặc cờ missing; đánh giá trên Validation/cross-validation
Km = 4.294.967.295	Giá trị sentinel/bất thường	Đối chiếu nguồn; đặt kiểm tra ngưỡng; không thay tùy tiện
Mẫu Train-Validation-Test ngẫu nhiên	Khả năng phân phối xe hiếm không cân	Stratified bins theo giá và kiểm thử ngoài thời điểm thu thập
year/engine_liters có NaN	Thiếu dữ liệu nguồn	Giữ NaN CatBoost; bổ sung kiểm tra và chứng từ nguồn
run.py cho chọn brand + grade bất kỳ	Có thể nhập tổ hợp xe không tồn tại	Lọc Grade theo Brand khi hiển thị, kiểm tra year/engine plausibility
parse_price có np.na	Có thể crash ở tin "Liên hệ"	Đổi thành np.nan và tạo unit tests
best_iteration=1999 của 2000	Chưa có dấu hiệu tối ưu số cây	Tune trên validation hoặc CV, không dùng Test để quyết định

12.2. Lộ trình cải thiện có kiểm chứng

• **Giai đoạn 1 - chất lượng dữ liệu:** làm rõ fuel_consumption, lọc sentinel mileage, chuẩn hóa Grade, đối chiếu giá và rút trích phiên bản nếu có dữ liệu.

• **Giai đoạn 2 - thiết kế đánh giá:** khóa một Test mới, kiểm thử theo giá/hãng, thêm thời gian và nhóm tránh gần trùng tin.

• **Giai đoạn 3 - tinh chỉnh CatBoost:** cân nhắc depth, regularization, iterations, target log1p hoặc loss robust (Huber/MAE) trên validation; chỉ chấp nhận khi các nhóm được cải thiện.

• **Giai đoạn 4 - triển khai:** hạn chế input ngoài miền dữ liệu; hiển thị độ tin cậy và cảnh báo nếu xe quá hiếm hoặc giá kỳ vọng bất thường.

12.3. Khả năng mở rộng dữ liệu

Nếu thu thập thêm tin đăng, cần nguồn có quyền sử dụng, ngày đăng, thị trường nhất quán, định dạng giá cùng đơn vị và chính sách xử lý bản ghi trùng. Tuyệt đối không ghép dữ liệu ngoại quốc (USD, năm, phân khúc khác) trực tiếp rồi báo cáo rằng mô hình Việt Nam đã cải thiện, nếu chưa xác thực phân phối và tỷ giá. Không dùng dữ liệu mới để sửa "bí mật" tập Test đã chốt.

13. KẾT LUẬN

Dự án đã hoàn thiện pipeline từ CSV đến CLI dự đoán, có báo cáo Train/Validation/Test, phân tích theo hãng và phân khúc giá, biểu đồ tầm quan trọng của feature và danh sách trường hợp dự đoán lệch. CatBoost là một lựa chọn phù hợp về mặt cấu trúc dữ liệu; mô hình hiện tại đã đạt R^2 Test = 0,9568, MAE Test = 112,08 triệu VNĐ và 85,30% mẫu nằm trong sai số ±20%.

Không nên diễn giải R^2 thành "accuracy 95,68%". Sai số tuyệt đối còn lớn ở một số xe trên 2 tỷ; những giá trị fuel_consumption bị suy diễn và mileage bất thường cần được nêu minh bạch. Điều này không làm mất giá trị nghiên cứu; ngược lại cho thấy quá trình phân tích có khả năng nhận diện vấn đề dữ liệu và đánh giá giới hạn của sản phẩm.

Thông điệp bảo vệ: Không tối ưu số điểm bằng mọi giá; cần chứng minh dữ liệu đã được biến đổi đúng, chỉ số đo được giải thích đúng, và ứng dụng có giới hạn sử dụng rõ ràng.

14. THAM KHẢO

[R1] Prokhorenkova et al. (2018). CatBoost: unbiased boosting with categorical features. NeurIPS 31.
https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html

[R2] CatBoost Documentation. Categorical features / transforming categorical to numerical.
https://catboost.ai/docs/en/features/categorical-features

[R3] CatBoost Documentation. Missing values processing.
https://catboost.ai/docs/en/concepts/algorithm-missing-values-processing

[R4] CatBoost Documentation. Common training parameters (Plain, SymmetricTree, nan_mode).
https://catboost.ai/docs/en/references/training-parameters/common

[R5] CatBoost Documentation. Feature importance / PredictionValuesChange.
https://catboost.ai/docs/en/concepts/fstr

[R6] Nguyễn Thái Cường và cộng sự (2025). Phân tích và dự báo giá bán xe ô tô đã qua sử dụng bằng phương pháp hồi quy tuyến tính.
https://vjol.info.vn/hunre/vi/article/view/142700/

15. ĐỊNH DANH CÁC NGUỒN NỘI BỘ

ZIP cung cấp: **Old_Car_Price_AI_zip(1).zip**. Các đường dẫn đã đọc: **data/raw/car_detail.csv**, **data/processed/cars_clean_v2.csv**, **data_check.py**, **preprocess.py**, **train.py**, **evaluate.py**, **run.py**, **models/catboost_car_price.cbm**, **models/feature_columns.json** và các file CSV/JSON trong **reports/**.

Các giá trị trong biểu đồ được tạo từ những CSV/JSON đã lưu (không làm lại việc train). Các giá trị như 85,30% trong ngưỡng ±20%, MAPE 12,66%, tỷ lệ sai số thuộc nhóm giá cao là phép tính bổ sung trực tiếp trên test_predictions.csv.